



- Name : Harshvardhan Patil
- Branch : AI&DS
- Roll No : SY B 23
- Batch : SY B2

Aim

To perform numerical computations using the NumPy library such as array creation, mathematical operations, statistical analysis, and advanced computations like matrix operations and solving equations.

Theory

1. Explain the role of NumPy in the Python scientific computing ecosystem.

NumPy plays an important role in Python scientific computing by providing support for large and efficient numerical calculations. It offers powerful data structures like arrays and matrices which are faster and more efficient than Python lists. NumPy is widely used in data science, engineering, and research because it provides many built-in mathematical functions. It also acts as a base library for other tools like Pandas, Matplotlib, and Scikit-learn, making it very important in the scientific computing ecosystem.

2. Why is NumPy considered faster than pure Python computations?

NumPy is faster than pure Python because it uses optimized C code in the background. It performs operations using vectorization, which means calculations are applied to the entire array at once instead of using loops. This reduces execution time and improves performance. Also, NumPy stores data in a continuous memory block, which makes processing more efficient compared to Python lists.

3. What types of mathematical operations can be performed using NumPy?

NumPy allows us to perform many types of mathematical operations such as basic arithmetic operations like addition, subtraction, multiplication, and division. It also supports advanced operations like trigonometric functions (sin, cos), logarithmic functions, exponential functions, and statistical operations like mean, median, variance, and standard deviation. It can also perform matrix operations like matrix multiplication and transpose.

4. Explain indexing and slicing in NumPy arrays.

Indexing in NumPy is used to access specific elements of an array by their position. For example, we can access a single value using its index number. Slicing is used to access a group of elements from an array by specifying a range. It allows us to extract a part of the array easily. NumPy also supports multi-dimensional indexing, which means we can access elements from rows and columns in a matrix.

5. How does NumPy integrate with libraries like Pandas, Matplotlib, and Scikit-learn?

a. Pandas: NumPy is the base of Pandas, and all Pandas data structures like DataFrame and Series are built using NumPy arrays. It helps Pandas perform fast data operations.

b. Matplotlib: NumPy is used with Matplotlib to create graphs and visualizations. Data stored in NumPy arrays can be directly used for plotting graphs.

c. Scikit-learn: Scikit-learn uses NumPy arrays to perform machine learning operations. All input data for models is usually in the form of NumPy arrays, making computations fast and efficient.

```
In [ ]: import numpy as np
import time

# BASIC OPERATIONS

# 1. Create NumPy array from list
data = [10, 20, 30, 40, 50]
arr = np.array(data)

print("NumPy Array:")
print(arr)

# 2. Perform basic arithmetic operations
print("\nArithmetic Operations:")
print("Addition:", arr + 5)
print("subtraction:", arr - 2)
print("Multiplication:", arr * 2)
print("Division:", arr / 2)

# 3. Find sum and mean
print("\nSum:", np.sum(arr))
print("Mean:", np.mean(arr))

# 4. Reshape a one-dimensional array
reshaped_arr = arr.reshape(5, 1)
print("\nReshaping Array:")
```

```

print(reshaped_arr)

# 5. Access specific elements using indexing
print("\nIndexing Elements:")
print("First element:", arr[0])
print("Last element:", arr[-1])
print("Elements from index 1 to 3:", arr[1:4])

# MODERATE OPERATIONS

# 6. Matrix addition and multiplication
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("\nMatrix Addition:")
print(A + B)

print("\nMatrix Multiplication:")
print(np.dot(A, B))

# 7. Generate random numbers
random_array = np.random.randint(1, 100, size=(2, 2))

print("\nRandom Array:")
print(random_array)

# 8. Compute variance and standard deviation
print("\nVariance:", np.var(arr))
print("Standard Deviation:", np.std(arr))

# 9. Use NumPy functions for mathematical operations
math_arr = np.array([0, np.pi/2, np.pi])
print("\nNumPy Mathematical Functions (sin, cos, tan):")
print("Sine of math_arr:", np.sin(math_arr))
print("Cosine of math_arr:", np.cos(math_arr))
print("Tangent of math_arr (first two elements):", np.tan(math_arr[:2]))

# 10. Compare Python lists and NumPy arrays (performance)
print("\nPerformance Comparison (Python list vs NumPy array):")
size = 1000000
python_list = list(range(size))
numpy_array = np.arange(size)

start_time = time.time()
python_list_result = [x + 1 for x in python_list]
end_time = time.time()
print(f"Python list addition took: {end_time - start_time:.6f} seconds")

start_time = time.time()
numpy_array_result = numpy_array + 1
end_time = time.time()
print(f"NumPy array addition took: {end_time - start_time:.6f} seconds")

```

ADVANCED OPERATIONS

11. Vectorized operation on a large array

```
large_arr = np.arange(1, 1000000)
result = large_arr * 2
print("\nVectorized Operation Completed (optimized large dataset analysis)")
```

12. Solve system of linear equations

$2x + y = 5$

$x + y = 3$

```
coeff = np.array([[2, 1], [1, 1]])
```

```
const = np.array([5, 3])
```

```
solution = np.linalg.solve(coeff, const)
```

```
print("\nSolution of Linear Equations (x, y):")
```

```
print(solution)
```

13. Broadcasting operation

```
arr1 = np.array([1, 2, 3])
```

```
arr2 = np.array([[10], [20], [30]])
```

```
broadcast_result = arr1 + arr2
```

```
print("\nBroadcasting Result:")
```

```
print(broadcast_result)
```

NumPy Array:
[10 20 30 40 50]

Arithmetic Operations:
Addition: [15 25 35 45 55]
subtraction: [8 18 28 38 48]
Multiplication: [20 40 60 80 100]
Division: [5. 10. 15. 20. 25.]

Sum: 150
Mean: 30.0

Reshaping Array:
[[10]
 [20]
 [30]
 [40]
 [50]]

Indexing Elements:
First element: 10
Last element: 50
Elements from index 1 to 3: [20 30 40]

Matrix Addition:
[[6 8]
 [10 12]]

Matrix Multiplication:
[[19 22]
 [43 50]]

Random Array:
[[65 67]
 [34 8]]

Variance: 200.0
Standard Deviation: 14.142135623730951

NumPy Mathematical Functions (sin, cos, tan):
Sine of math_arr: [0.00000000e+00 1.00000000e+00 1.2246468e-16]
Cosine of math_arr: [1.0000000e+00 6.123234e-17 -1.0000000e+00]
Tangent of math_arr (first two elements): [0.00000000e+00 1.63312394e+16]

Performance Comparison (Python list vs NumPy array):
Python list addition took: 0.030307 seconds
NumPy array addition took: 0.002474 seconds

Vectorized Operation Completed (optimized large dataset analysis)

Solution of Linear Equations (x, y):
[2. 1.]

Broadcasting Result:

```
[[11 12 13]
 [21 22 23]
 [31 32 33]]
```

Conclusion

In this assignment, we understood the importance of NumPy in numerical computation and scientific computing. We learned how NumPy provides fast and efficient operations compared to normal Python by using optimized functions and vectorization. We also explored different types of mathematical operations, indexing and slicing techniques, and how NumPy works with other libraries like Pandas, Matplotlib, and Scikit-learn. Overall, NumPy is a powerful library that makes numerical calculations easier, faster, and more efficient, which is very useful in data analysis and scientific applications.